

Table of Contents

Week 11 — Abstraction: Interfaces

The Fourth Pillar (Part 2): A Pure Contract, No Shared Behavior

Lesson Overview

Hour Plan

Recap of Week 10 (4 min)

Part 1 — What Is an Interface? A Pure Contract (11 min)

Open with a story (3 min)

Definition

Three Real-World “Pure Contract” Scenarios

What an Interface Is NOT

Part 2 — Interface vs Abstract Class: Side-by-Side (10 min)

Open with a question (2 min)

Side-by-Side Comparison

The Decision Rule

Part 3 — One Class, Many Interfaces (11 min)

Open with a question (2 min)

The Payoff: Combining Inheritance and Multiple Interfaces

Tracing the Rule With a Second Example

Part 4 — Why Interfaces Matter: Unrelated Classes, Shared Behavior (9 min)

Open with a comparison (2 min)

Without Interfaces (forced into awkward, repeated code)

With Interfaces (the OOP way)

Part 5 — Java as Illustration (8 min)

Reading Interfaces as a Concept

One Class, Multiple Interfaces (the Duck Example)

Polymorphism Across Unrelated Classes

Reading as Plain English

Test Checkpoint

Common Misconceptions

Extension Challenges

-  Basic — Interface or Abstract Class?
-  Intermediate — Trace the Capabilities

● Advanced — Design a Multi-Interface Class

Lesson Summary

Homework (Prep for Object Relationships — HAS-A, Week 12)

Week 11 — Abstraction: Interfaces

The Fourth Pillar (Part 2): A Pure Contract, No Shared Behavior

Date: Wednesday, 12 August 2026 | **Time:** 6:30 PM – 7:30 PM | **Course:** Object-Oriented Programming Concepts

Lesson Overview

Item	Detail
Topic	Abstraction — interfaces as a pure contract, and how unrelated classes can share behavior through interfaces without inheritance
Date & Time	Wednesday, 12 August 2026 · 6:30 PM – 7:30 PM
Duration	1 hour
Format	Online (Google Meet) — Concept explanation + real-world discussion + light illustration
Prerequisites	Week 6 (Inheritance), Week 9 (Polymorphism), Week 10 (Abstract Classes — “a parent with no sensible default”)
Focus	OOP concept understanding — Java used only as illustration
Outcome	Students can define an interface as a “pure contract” with zero shared behavior, explain why a class can implement many interfaces but extend only one class, and explain how interfaces let completely unrelated classes share polymorphic behavior

Hour Plan

00:00 – 00:04	Recap of Week 10 + Homework check
00:04 – 00:15	Part 1 – What Is an Interface? A Pure Contract
00:15 – 00:25	Part 2 – Interface vs Abstract Class: Side-by-Side
00:25 – 00:36	Part 3 – One Class, Many Interfaces
00:36 – 00:45	Part 4 – Why Interfaces Matter: Unrelated Classes, Shared Behavior

 **Google Meet tip:** Keep *Week11_Summary_Examples.md* open in a second tab — it has a completely different set of real-world stories (job qualifications and remote-controlled devices) ready for every part of this lesson.

Recap of Week 10 (4 min)

Ask students verbally:

1. "What is an abstract class, in one sentence?"
→ *A parent class that cannot be built directly — it exists only to define a category and force children to implement certain actions.*
2. "From homework — your `PaymentMethod` abstract class had ONE regular, shared method (`printReceipt()`) and ONE abstract method (`processPayment()`). Why did `printReceipt()` NOT need to be abstract?"
→ *Because printing "Receipt: \$[amount] paid" can genuinely be the exact same working behavior for every payment type — there was no need to force each child to redefine it.*
3. "What if a class needed to define a contract that has ZERO shared behavior — where literally every single action is left for children to fully define?"
→ *That's exactly what today's topic is: an interface.*
4. Transition question: "And here's something new — what if a `CardPayment` object also needed to follow a completely UNRELATED contract, like being 'Refundable,' at the same time it follows the `PaymentMethod` contract? Can a class follow more than one contract at once? That is today's biggest new idea."

Part 1 — What Is an Interface? A Pure Contract (11 min)

Open with a story (3 min)

"Think about a USB port. Every USB device — a mouse, a keyboard, a flash drive, a phone charger — plugs into the exact same shaped port. USB is not a physical object you can hold. It's a STANDARD — a published contract that says: 'if your device follows these exact specifications, it WILL fit and WILL work in any USB port.'

USB itself has no behavior of its own. It doesn't move data, charge a battery, or do anything by itself. It ONLY defines the rules that every device must follow. That is exactly what an interface is in OOP."

Definition

An **interface** is a pure contract: it declares WHAT actions a class must provide, with absolutely NO shared behavior of its own — every single method is a requirement, not a default. Any class that “implements” an interface is making a promise: “I will provide a working version of every action this contract demands.”

Three Real-World “Pure Contract” Scenarios

Work through each one. Ask: “Does this standard actually DO anything by itself, or does it only describe what’s required?”

SCENARIO 1 – PAYMENT (continuing from Week 10)

Interface:	Payable
Requires:	pay(amount) – no default, every implementer must define their own way of paying

Implemented by:	Invoice, Employee, Subscription (completely unrelated classes – none of them extend a shared parent)
-----------------	---

Why an interface, not an abstract class?

There is NOTHING shared between how an Invoice gets paid, how an Employee gets paid, and how a Subscription gets paid. Zero shared behavior – 100% pure requirement.

SCENARIO 2 – DRIVING (vehicle behavior)

Interface:	Drivable
Requires:	accelerate(), brake() – no defaults

Implemented by:	Car, Motorbike (both extend Vehicle already – Drivable is an ADDITIONAL contract on top of that)
-----------------	---

Why an interface?

“Being drivable” is a capability, not a shared family – it's simply a promise that the action exists.

SCENARIO 3 – COMPARING (sorting a list)

Interface:	Comparable
------------	------------

Implemented by: Student, Product, Employee
 (totally unrelated classes – the ONLY thing
 they share is "I can be compared to another
 one of my own kind")

Why an interface?

A sorting tool doesn't need to know anything about
Students, Products, or Employees – it only needs to trust
that each one has a working compareTo().

What an Interface Is NOT

NOT AN INTERFACE:

A class that provides SOME shared, working behavior
→ that's an abstract class (Week 10) – interfaces provide
ZERO shared behavior, no exceptions

A description of what an object IS (its identity/category)
→ that's still inheritance (IS-A) – a Car IS-A Vehicle.
An interface instead describes a CAPABILITY a class HAS
(Car implements Drivable – "Car CAN drive")

AN INTERFACE:

A pure list of required actions, with NO shared behavior,
that any unrelated class can promise to follow.

Part 2 — Interface vs Abstract Class: Side-by-Side (10 min)

Open with a question (2 min)

"We now have TWO tools that both stop you from building a plain object directly and both force children to implement something. What's actually different between them?"

Side-by-Side Comparison

ABSTRACT CLASS (Week 10)

INTERFACE (today)

CAN IT HAVE SHARED, WORKING BEHAVIOR?

Yes – regular methods with real instructions are allowed

No – a pure contract has zero shared behavior (every method is required)

HOW MANY CAN A CLASS USE AT ONCE?

Only ONE – a class can extend only one abstract class

MANY – a class can implement several interfaces at once

WHAT RELATIONSHIP DOES IT DESCRIBE?

IS-A – Circle IS-A Shape

CAN-DO / HAS-CAPABILITY – Car CAN implement Drivable

WHEN TO USE?

Children share a real family AND some real, common behavior

Completely unrelated classes need to promise the SAME capability, with no shared family or behavior at all

The Decision Rule

ASK YOURSELF:

"Do these classes share a real family AND at least some real, working behavior in common?"

YES → abstract class (Shape → Circle, Rectangle)

"Do these classes have NOTHING in common except one required capability, and they may not even be related by inheritance at all?"

YES → interface (Invoice, Employee, Subscription → all implement Payable, despite being unrelated)

Part 3 — One Class, Many Interfaces (11 min)

Open with a question (2 min)

"A duck IS-A Animal. A duck can also swim, and a duck can also fly. Swimming and flying aren't 'family' relationships — they're separate CAPABILITIES a duck happens to have. Could a duck 'extend' both Swimmable and Flyable as parent classes, the way it extends Animal?"

Students will realize: A class can only extend ONE parent class (Week 6 rule), but a duck clearly needs multiple separate capabilities. This is exactly why interfaces allow MULTIPLE implementation.

The Payoff: Combining Inheritance and Multiple Interfaces

```
class Duck extends Animal implements Swimmable, Flyable {  
    // ONE parent class (the real family - IS-A Animal)  
    // TWO interfaces (separate capabilities - CAN swim, CAN fly)  
}
```

WHY THIS WORKS:

Duck has exactly ONE real family tree - it extends Animal, and only Animal. That single-parent rule from Week 6 still applies.

But Duck can PROMISE as many separate capabilities as it actually has. Swimmable and Flyable are not "family" - they're independent contracts Duck chooses to follow.

A Fish might also implement Swimmable (it CAN swim too), without needing ANY family relationship to Duck at all.

Tracing the Rule With a Second Example

```
class Car extends Vehicle implements Drivable, Insurable {  
    // ONE parent - extends Vehicle (real family, IS-A)  
    // TWO interfaces - implements Drivable, Insurable  
    // (separate, unrelated capabilities Car happens to have)
```



```
}

class House implements Insurable {
    // NO parent class needed at all
    // ONE interface - implements Insurable
    // (House has NOTHING to do with Vehicle or Car, but it
    // shares the SAME "can be insured" capability)
}
```

THE KEY INSIGHT:

Car and House share ZERO family relationship – one is a Vehicle, the other isn't related to Vehicle at all. But BOTH can implement Insurable, because "being insurable" is a capability, not a family trait.

This is the real power of interfaces: completely different, unrelated classes can still be treated the SAME way by any code that only cares about the ONE shared capability.

Part 4 — Why Interfaces Matter: Unrelated Classes, Shared Behavior (9 min)

Open with a comparison (2 min)

"Imagine an accounting system that needs to process payment for Invoices, Employee salaries, and Subscription renewals — three completely unrelated classes. Without interfaces, how would one loop handle all three?"

Without Interfaces (forced into awkward, repeated code)

```
processInvoicePayment(invoice);
processEmployeePayment(employee);
processSubscriptionPayment(subscription);
// Three separate function calls, three separate code paths,
// because these classes share no common type at all.
```

With Interfaces (the OOP way)

```
Payable[] paymentsDue = { invoice, employee, subscription };

for (Payable p : paymentsDue) {
    p.pay(amount);           // ONE loop – works for ALL of them,
                             // even though they are otherwise
                             // completely unrelated classes
}
```

THE PRINCIPLE:

Interfaces let POLYMORPHISM (Week 9) work across classes that have NOTHING to do with each other in the inheritance tree. As long as each class implements the same interface, one loop, one shared type, treats them identically – even though Invoice, Employee, and Subscription would never be related by a shared parent class.

This is the payoff of a "pure contract": it connects classes purely by CAPABILITY, not by FAMILY.

Part 5 — Java as Illustration (8 min)

Reminder: Java is just how we write these concepts down. Focus on the concept, not the syntax.

Reading Interfaces as a Concept

```
// INTERFACE – a pure contract, zero shared behavior
public interface Payable {
    void pay(double amount); // no body – just the requirement
}

// COMPLETELY UNRELATED CLASSES all implementing the same contract
public class Invoice implements Payable {
    private String invoiceId;

    public Invoice(String invoiceId) {
        this.invoiceId = invoiceId;
    }
}
```

```

        @Override    public void pay(double amount) {
            System.out.println("Invoice " + invoiceId + " paid: $" + amount);    }    }
        public class Employee implements Payable {    private String name;
    public Employee(String name) {        this.name = name;    }                @Override
    public void pay(double amount) {
        System.out.println(name + "'s salary deposited: $" + amount);    }    }
        public class Subscription implements Payable {    private String plan;
    public Subscription(String plan) {        this.plan = plan;    }                @Override
    public void pay(double amount) {
        System.out.println(plan + " subscription renewed for: $" + amount);    }    }

```

One Class, Multiple Interfaces (the Duck Example)

```

public interface Swimmable {
    void swim();
}

public interface Flyable {
    void fly();
}

public class Duck extends Animal implements Swimmable, Flyable {
    @Override
    public void swim() {
        System.out.println("Duck paddles across the pond");
    }

    @Override
    public void fly() {
        System.out.println("Duck flaps its wings and takes off");
    }
}

```

Polymorphism Across Unrelated Classes

```

Payable[] paymentsDue = {
    new Invoice("INV-001"),
    new Employee("Sokha"),
    new Subscription("Premium Plan")
};

for (Payable p : paymentsDue) {    // "p" has the LABEL Payable
    p.pay(100);                    // each unrelated class runs its own version
}

```

Reading as Plain English

CONCEPT TRANSLATION TABLE

Java code	OOP concept meaning
<code>interface Payable</code>	A pure contract – no shared behavior, only a requirement
<code>void pay(double amount);</code> (inside an interface)	No body – every implementer MUST write their own version
<code>implements Payable</code>	A promise: "I will provide a working version of every action this contract demands"
<code>extends Animal implements Swimmable, Flyable</code>	ONE real family (Animal) + TWO separate capabilities

Test Checkpoint

- ☐ Can define an interface as a "pure contract" with zero shared behavior, using the USB-standard analogy
- ☐ Can explain the difference between an abstract class (some shared behavior allowed) and an interface (no shared behavior at all)
- ☐ Can explain why a class can extend only ONE parent class but implement MANY interfaces
- ☐ Can explain, using the Duck example, the difference between a real family relationship (IS-A) and a capability (CAN-DO)
- ☐ Can explain how interfaces let completely unrelated classes be treated the same way in a single polymorphic loop

Common Misconceptions

Misconception	Clarification
"Interfaces and abstract classes are basically the same thing"	They solve similar problems (forcing children to implement something) but differ completely on shared behavior: abstract classes CAN have real, working methods; interfaces have ZERO shared behavior — every method is a pure requirement.

only extend one parent"	False — this is the entire point of today's lesson. A class can implement as MANY interfaces as it needs, while still extending only one parent class.
"Implementing an interface is a family (IS-A) relationship"	No — implementing an interface describes a CAPABILITY (CAN-DO), not family membership. A Car IS-A Vehicle (inheritance), but Car CAN BE Drivable (interface) — these are different kinds of relationships.
"If two classes implement the same interface, they must be related by inheritance too"	Not at all — Invoice, Employee, and Subscription share zero family relationship but can all implement Payable. The interface is the ONLY thing connecting them.
"Interfaces are just a stricter version of abstract classes, so why not always use abstract classes instead?"	Because abstract classes force a real family relationship (single inheritance), while interfaces let unrelated classes share a capability without forcing them into an artificial family tree.

Extension Challenges

● Basic — Interface or Abstract Class?

For each scenario below, decide whether an interface or an abstract class fits better. Justify your answer using the decision rule from Part 2.

1. `Circle`, `Rectangle`, `Triangle` — all share the real family "Shape" AND some common behavior like `describe()`
2. `Invoice`, `Employee`, `Subscription` — completely unrelated classes that all need a `pay()` action, with no shared behavior
3. `Book`, `Movie`, `Podcast` — completely unrelated classes that all need a `getRating()` action, with no shared family
4. `SavingsAccount`, `CheckingAccount` — both share the real family "BankAccount" AND some common behavior like `printStatement()`

● Intermediate — Trace the Capabilities

Given: `class Smartphone extends Device implements Callable, Cameraable` and `class Tablet extends Device implements Cameraable`.

1. What real family do both Smartphone and Tablet share?
2. What capability does Smartphone have that Tablet does NOT?
3. Could a completely unrelated class, like `SecurityCamera` (which does NOT extend Device), also implement `Cameraable`? Explain why or why not.

● Advanced — Design a Multi-Interface Class

Design (in plain English, no code required) a `SmartWatch` class that:

- Extends ONE real parent class of your choice (e.g., `Device` or `Wearable`)
- Implements AT LEAST TWO separate interfaces representing capabilities (e.g., `Callable` , `HeartRateMonitorable` , `Waterproof`)

For each interface you choose, explain why it makes sense as a CAN-DO capability rather than a real family (IS-A) relationship. Then explain what would go wrong if you tried to make all of these capabilities into abstract classes instead of interfaces.

Lesson Summary

TODAY'S KEY TAKEAWAYS

1. An INTERFACE is a pure contract – zero shared behavior, every method is a required promise, no defaults exist.
2. ABSTRACT CLASS vs INTERFACE:
Abstract class → CAN have shared, working behavior,
but a class can extend only ONE
Interface → NO shared behavior at all, but a class can
implement MANY at once
3. Inheritance (extends) describes a real FAMILY – IS-A.
Implementing an interface describes a CAPABILITY – CAN-DO.
A class can have ONE family but MANY capabilities.
4. Interfaces let completely UNRELATED classes (no shared parent at all) be treated identically by polymorphic code, as long as they share the same capability.
5. THE DECISION RULE: shared family + shared real behavior → abstract class. Shared capability only, possibly across totally unrelated classes → interface.

Homework (Prep for Object Relationships — HAS-A, Week 12)

Design the following system in plain English — code is optional (bonus only):

Interface `Chargeable` implemented by unrelated classes `Smartphone`, `ElectricCar`, `Laptop` :

1. `Chargeable` (interface): requires `charge()` — no default, every implementer defines their own charging behavior
2. `Smartphone` implements `Chargeable` : `charge()` — "Charging via USB-C cable"
3. `ElectricCar` implements `Chargeable` : `charge()` — "Charging at a charging station"
4. `Laptop` implements `Chargeable` : `charge()` — "Charging via power adapter"

Then design a `Chargeable[]` array holding one of each, and write out — in plain English — what a single loop calling `charge()` on every element would print.

Concept to carry forward: Notice that your `ElectricCar` needs a `Battery` object to actually track its charge level — the car doesn't just calculate a formula, it genuinely CONTAINS a separate Battery object as one of its parts. A class containing another object as a real part of itself (not a parent-child family relationship) is the seed of our next topic: **Object Relationships — the HAS-A relationship** (Week 12).

Week 11 of 16 | Object-Oriented Programming Concepts | @KhmerSide | syuthd.com